DISEÑO DIGITAL 1.

TÍTULO DE LA ACTIVIDAD: **Ejercicios de repaso de la metodología clásica de síntesis de circuitos combinacionales e introducción al modelado con VHDL**	**CÓDIGO:** **ATGNP1**

FECHA:			
NOMBRE:		APELLIDOS:	

MODALIDAD:	Ejercicios. Aprendizaje cooperativo.	TIPO:	No Presencial	DURACIÓN:	120 minutos

CALENDARIO:	Previo a Sesión Presencial S3	REQUISITOS:	Conocimientos adquiridos en Electrónica 2. Tutorial VHDL y ModelSim.

CRITERIO DE ÉXITO:	

COMENTARIOS E INCIDENCIAS:

TIEMPO DEDICADO:	minutos	AUTOEVALUACIÓN: [entre 0 y 10 puntos]	No procede

Introducción

En esta actividad realizará cinco ejercicios:

- Un ejercicio de repaso de la metodología de síntesis de circuitos combinacionales a partir de tablas de verdad.
- Cuatro ejercicios relacionados con el modelado VHDL de sistemas combinacionales

Aunque la actividad está clasificada en la modalidad de aprendizaje cooperativo, la dinámica de trabajo será la siguiente:

- Realice los ejercicios individualmente. Anote los resultados sobre el propio enunciado. Si tiene dudas puede consultar en el foro de dudas (preferentemente) o directamente a su profesor.
- Realice una puesta en común con su grupo de 4 estudiantes. Debe llevarse a cabo una discusión de los apartados en los que se encuentren discrepancias. Las dudas que subsistan deben ser anotadas y llevadas a la sesión presencial S3, en la cual serán resueltas por el profesor.

PARTE I. Ejercicio 1[1]

Los *displays* de 7 segmentos cuentan con 7 LEDs (identificados con las letras que van de la *a* a la *g*), que permiten componer, entre otros símbolos, la forma de los dígitos decimales (figura 1). En la figura 1b se muestra, a modo de ejemplo, cómo se puede dibujar el 1.

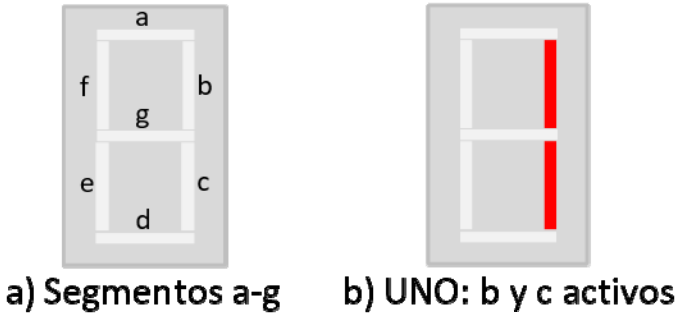


Figura 1.- *Displays de 7 segmentos*

Un decodificador BCD a 7 segmentos es un circuito que se utiliza para representar los dígitos decimales en un *display de 7 segmentos*. La entrada del decodificador es un dígito BCD (cuatro bits), que indica el número que se desea representar, y la salida controla los segmentos del *display* que deben encenderse para el dígito en cuestión (7 bits). En la figura 2 se muestra la interfaz de un decodificador BCD a 7 segmentos.

[1] Importante: La solución de este ejercicio está disponible al final de este documento. Este es un ejercicio de repaso. Si no puede resolver un apartado de forma directa, no pierda más tiempo y vaya directamente a revisar la solución.

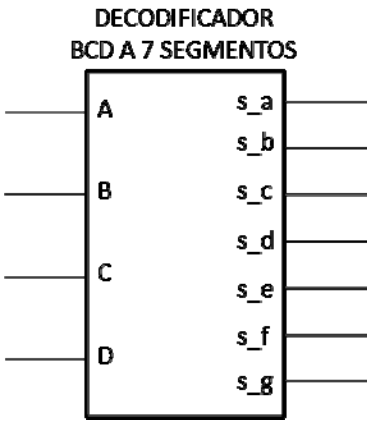


Figura 2.- Símbolo de un decodificador BCD a 7 segmentos

En la figura 3 se muestran las combinaciones de LEDs que permiten dibujar los dígitos decimales.

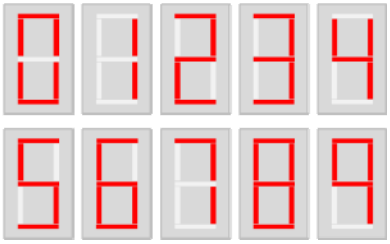


Figura 3.- Dígitos decimales

1. Complete la tabla de verdad adjunta para que represente el funcionamiento de un decodificador BCD a 7 segmentos con las salidas activas a nivel alto (las salidas correspondientes a un segmento que se deba encender para dibujar el dígito se ponen a 1).

D	C	B	A	S_A	S_B	S_C	S_D	S_E	S_F	S_G
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0							
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1							
0	1	1	0	1	0	1	1	1	1	1
0	1	1	1							
1	0	0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	0	0	1	1
Resto				X	X	X	X	X	X	X

2. Exprese en hexadecimal el valor de las salidas del decodificador BCD a 7 segmentos para cada uno de los dígitos decimales.

   Ejemplo:

D	C	B	A	S_A	S_B	S_C	S_D	S_E	S_F	S_G
0	0	0	0	1	1	1	1	1	1	0



DÍGITO	0	1	2	3	4	5	6	7	8	9
HEXA	0x7E									

Nota: Antes de realizar los siguientes apartados del ejercicio, compruebe, en la solución que se adjunta al final de la actividad, que ha completado correctamente la tabla de verdad del decodificador.

3. Extraiga de la tabla de verdad:

   a. La forma canónica disyuntiva de S_E:

   b. La expresión de S_G como producto de maxitérminos (forma canónica conjuntiva):

4. Obtenga de la tabla de verdad del decodificador las siguientes funciones lógicas en su expresión resumida:

$$S_F = \sum_{(D,C,B,A)}($$

$$S_D = \prod_{(D,C,B,A)}($$

5. Averigüe a qué salidas del decodificador corresponden las siguientes funciones lógicas:

$$F_1 = \overline{C} + \overline{A} \oplus B$$

$$F_2 = B + D + \overline{A \oplus C}$$

6. En los apartados anteriores ha extraído la expresión correspondiente a 4 salidas e identificado, a partir de expresiones dadas, las de otras dos. Obtenga, para la salida que falta, la expresión algebraica con menor número de términos y simplifíquela aprovechando que la tabla de verdad del decodificador es incompleta (contiene indiferencias).

Ejercicio 2

Un decodificador binario es un circuito combinacional cuya entrada es un número de *n* bits, codificado en binario natural, y que dispone de 2^n bits de salida. Para cada combinación de entrada se activa un único bit de salida del decodificador, aquel cuyo *peso* se corresponde con el codificado en la entrada; a modo de ejemplo, si la combinación de entrada codifica el número 3, en la salida se activa el bit que tiene asociado el peso 3 y todos los demás bits de salida permanecen inactivos. En la figura 4 se muestra el modelo lógico de un decodificador binario de 2 bits con entradas y salidas activas a nivel alto.

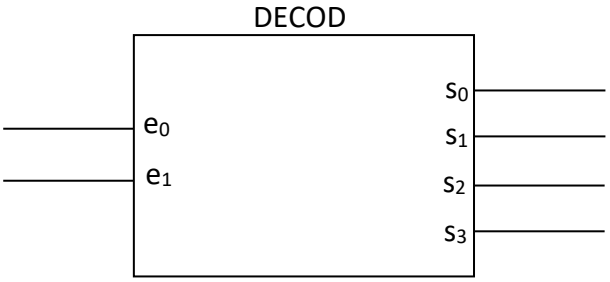


Figura 4.- Decodificador binario

1. Dibuje el símbolo y la tabla de verdad de un decodificador binario de 3 a 8 líneas (3 bits de entrada y 8 de salida) con entradas y salidas activas a nivel bajo.

El modelo de la interfaz de un sistema en VHDL se realiza construyendo una Declaración de Entidad, en la que se especifica:

- ✓ El nombre del sistema modelado.
- ✓ Las entradas y salidas (los puertos) de la interfaz del sistema, indicando su nombre, dirección y tipo de datos –el tipo de datos define implícitamente el número de bits de cada *puerto*.
- ✓ En los modelos VHDL sintetizables la Declaración de entidad viene precedida por dos *cláusulas de visibilidad* que permiten emplear los tipos de datos *std_logic* y *std_logic_vector*.

El siguiente código modela la interfaz del decodificador de la figura 4:

```
library ieee;
use ieee.std_logic_1164.all;

entity decod2a4 is
port(E: in     std_logic_vector(1 downto 0);
     S: buffer std_logic_vector(3 downto 0));
end entity;
```

- ✓ El sistema modelado se llama **decod2a4**

- ✓ **decod2a4** tiene dos *puertos*: el puerto de entrada (*in*) se llama **E** y es de dos bits, puesto que su tipo de datos, *std_logic_vector*, tiene un rango descendente (*downto*) de 1 a 0; el puerto de salida (*buffer*) se llama **S**, y es de cuatro bits (*std_logic_vector(3 downto 0)*).

2. Utilice ModelSim[2][3] para crear un fichero, de nombre **decod3a8n.vhd** y codificar en él el modelo VHDL de la interfaz del decodificador de 3 a 8 líneas con entradas y salidas activas a nivel bajo. Utilice las siguientes etiquetas:

   - ✓ **decod3a8n** para el nombre del modelo,
   - ✓ **n_E** para el *puerto* de entrada,
   - ✓ **n_S** para el *puerto* de salida.

El modelo de funcionamiento VHDL de un sistema digital se codifica en un Cuerpo de Arquitectura. El funcionamiento de los sistemas combinacionales puede expresarse empleando un *proceso con lista de sensibilidad*; estos procesos contienen sentencias que se ejecutan secuencialmente, durante el transcurso de una simulación, cada vez que hay *eventos* (cambios de valor) en cualquiera de las señales que forman parte de la *lista de sensibilidad* del proceso.

Para construir el código de un proceso que modele el funcionamiento de un sistema combinacional hay que tener en cuenta que:

- ✓ El proceso tiene que ser sensible a TODAS las entradas del combinacional.

- ✓ Dentro del proceso debe codificarse un algoritmo de ejecución secuencial que calcule, para TODAS las combinaciones de los bits de entrada, el valor que toma cada bit de salida.

El siguiente código modela el funcionamiento del decodificador de la figura 4:

[2] Cree un único proyecto, que puede llamarse, por ejemplo, proy_ATGNP1, y utilícelo para albergar en él los ficheros creados en los ejercicios 2 y 3. No olvide compilar cada fichero para así realizar un chequeo sintáctico.
[3] Alternativamente, puede utilizar un editor de texto tipo WordPad, en cuyo caso no podrá realizar el chequeo sintáctico.

```
architecture rtl of decod2a4 is
begin
  process(E)
  begin
    if E = "00" then
      S <= "0001";

    elsif E = "01" then
      S <= "0010";

    elsif E = "10" then
      S <= "0100";

    elsif E = "11" then
      S <= "1000";

    end if;
  end process;
end rtl;
```

Observe que:

- ✓ El Cuerpo de Arquitectura (unidad de código VHDL *secundaria*) en el que se define el proceso se llama **RTL** (etiqueta que alude al hecho de que es un modelo *sintetizable* automáticamente) y está asociado a la Declaración de Entidad (unidad de código VHDL *primaria*) **decod2a4** en la que se modela la interfaz del decodificador –por medio de esta asociación el Cuerpo de Arquitectura **RTL** *obtiene* visibilidad sobre los puertos declarados en la Entidad y puede utilizar sus nombres para definir el funcionamiento del sistema.

- ✓ El proceso que define el funcionamiento del decodificador es sensible a la entrada **E** (la única entrada del decodificador) y mediante una sentencia **IF-ELSIF** especifica el valor de los cuatro bits de salida para las cuatro combinaciones de los dos bits de entrada –cumple, por tanto, con las reglas de modelado de sistemas combinacionales enunciadas anteriormente.

El siguiente código muestra una versión alternativa para la descripción del funcionamiento del decodificador en la que se hace uso de una sentencia CASE:

```
process(E)
  begin
    case E is
      when "00" =>
        S <= "0001";
      when "01" =>
        S <= "0010";
      when "10" =>
        S <= "0100";
      when others =>
        S <= "1000";
    end case;
  end process;
```

3. Edite, en el fichero que contiene el modelo de la interfaz del decodificador de 3 a 8 líneas y a continuación del código de la Declaración de Entidad, el modelo de funcionamiento de dicho decodificador empleando sentencias IF o CASE (las que prefiera) –recuerde que las entradas y salidas de este decodificador deben ser activas a nivel bajo.

Ejercicio 3

1. Dibuje el símbolo y la tabla de verdad del sistema combinacional modelado por el siguiente código VHDL:

```
library ieee;
use ieee.std_logic_1164.all;
entity ctrl_inc is
port(ctrl:     in     std_logic;
     Dato_in:  in     std_logic_vector(1 downto 0);
     Dato_out: buffer std_logic_vector(1 downto 0));
end entity;

architecture rtl of ctrl_inc is
begin
  process(ctrl, Dato_in)
  begin
    if ctrl = '0' then
      Dato_out <= Dato_in;

    else
      case Dato_in is
        when "00" =>
          Dato_out <= "01";
        when "01" =>
          Dato_out <= "10";
        when "10" =>
          Dato_out <= "11";
        when others =>
          Dato_out <= "00";
      end case;
    end if;
  end process;
end rtl;
```

2. Deduzca y explique la función que realiza el modelo **ctrl_inc**

3. Utilice ModelSim[4] para crear un fichero, de nombre **ctrl_dec.vhd** y codificar en él el modelo VHDL de un sistema combinacional (**ctrl_dec**) que decrementa un dato de entrada de 2 bits (**Dato_in**) cuando su entrada **ctrl** vale 1 y deja pasar a la salida (**Dato_out**) el dato de entrada (el ya mencionado **Dato_in**) cuando **ctrl** vale 0.

[4] O, de forma alternativa, un editor de texto tipo WordPad.

Los modelos VHDL son simulables porque los *procesos* que codifican el funcionamiento se *ejecutan* durante el transcurso de una simulación cuando las señales a las que son sensibles (las que aparecen en su lista de sensibilidad) sufren *eventos* (cambian de valor). Cuando se simula un sistema combinacional, el proceso que modela su funcionamiento se ejecuta cuando cambia el valor de cualquiera de las entradas del combinacional, ya que dichas entradas son precisamente las señales a las que debe ser sensible el proceso.

Por ejemplo, si en una simulación se prueba **ctrl_inc** con las señales del cronograma de la figura 5, el proceso que modela su funcionamiento se ejecutaría en los siguientes instantes de tiempo:

- ✓ en 15 y 25 ns porque hay eventos en la señal **ctrl**,
- ✓ en 20, 30 y 50 ns porque hay eventos en la señal **Dato_in**,
- ✓ en 0 y 40 ns porque hay eventos en ambas señales.

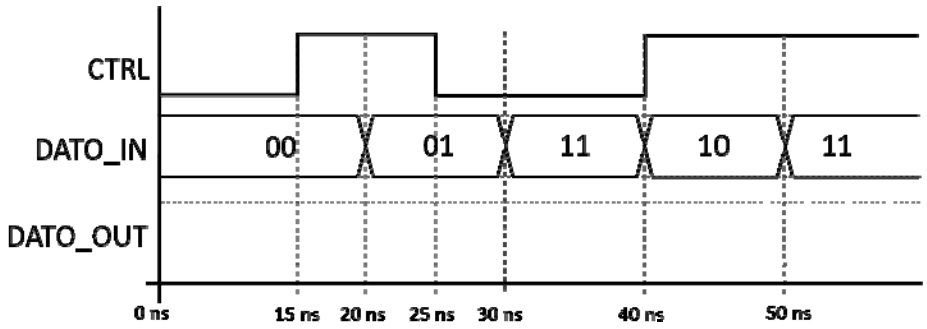


Figura 5.- Eventos en señales

Cada vez que el proceso se activa, calcula el valor de las salidas mediante la ejecución de las sentencias que contiene. Por ejemplo, en la simulación representada por el cronograma de la figura 5 el proceso se ejecutaría por primera vez en T = 0 ns, instante en el que la entrada **ctrl** vale 0 y la entrada **Dato_in** vale 00, por lo que el proceso asignaría a la salida (**Dato_out <= Dato_in**) el valor 00; tras ejecutarse, el proceso se suspende hasta la ocurrencia de un nuevo evento que, en el ejemplo, se produce en T = 15 ns, instante en el que se vuelve a ejecutar el proceso y recalcula el valor de salida –en este caso, con **ctrl** igual a 1 y **Dato_in** igual a 00, se asigna a **Dato_out** el valor 01. En la figura 6 se muestra el valor que toma **Dato_out** como consecuencia de estas dos ejecuciones del proceso.

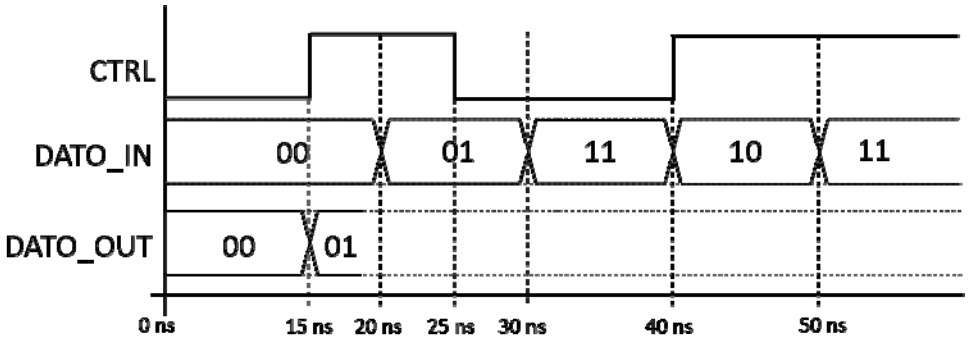


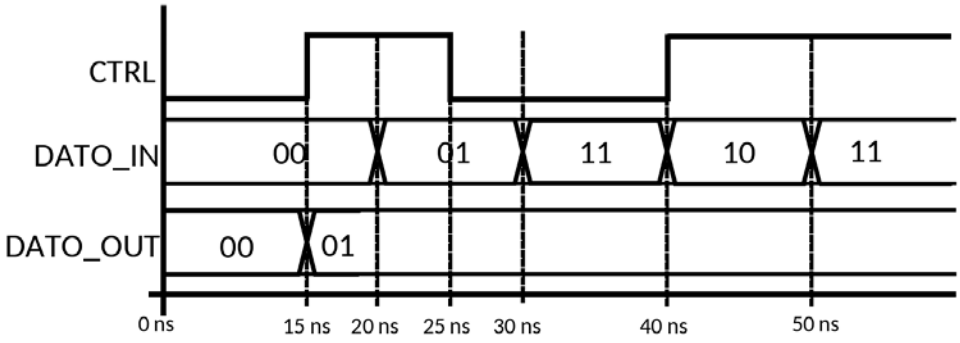
Figura 6.- Transcurso de la simulación

4. Rellene la tabla adjunta y complete el cronograma de la figura 6 (que para su comodidad se reproduce aquí, junto con el código del proceso que modela el funcionamiento de **ctrl_inc**), teniendo en cuenta el código del proceso y el valor de las entradas.

Tiempo de Simulación	ctrl	Dato_in	Dato_out
0 ns	0	00	00
15 ns	1	00	01

```
architecture rtl of ctrl_inc is
begin
  process(ctrl, Dato_in)
  begin
    if ctrl = '0' then
      Dato_out <= Dato_in;

    else
      case Dato_in is
        when "00" =>
          Dato_out <= "01";
        when "01" =>
          Dato_out <= "10";
        when "10" =>
          Dato_out <= "11";
        when others =>
          Dato_out <= "00";
      end case;
    end if;
  end process;
```



Ejercicio 4

El siguiente código VHDL corresponde al modelo de un sistema combinacional que invierte los bits de un dato de entrada, **Dato_in**, cuando la entrada de control (**ctrl**) vale 0, o *transpone* los pesos de los bits de **Dato_in** cuando **ctrl** vale 1.

```
library ieee;
use ieee.std_logic_1164.all;

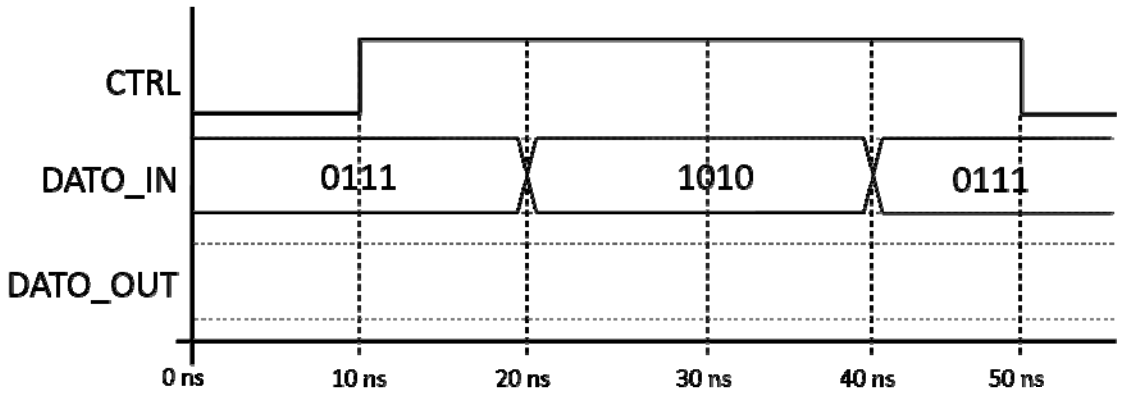
entity inv_trans is
port(ctrl:     in     std_logic;
     Dato_in:  in     std_logic_vector(3 downto 0);
     Dato_out: buffer std_logic_vector(3 downto 0));
end entity;

architecture rtl of inv_trans is
begin
  process(ctrl, Dato_in)
  begin
    if ctrl = '0' then
      Dato_out <=  not Dato_in;

    else
      Dato_out(0) <= Dato_in(3);
      Dato_out(1) <= Dato_in(2);
      Dato_out(2) <= Dato_in(1);
      Dato_out(3) <= Dato_in(0);
    end if;
  end process;
end rtl;
```

1. Rellene la tabla adjunta y complete el cronograma de la figura, teniendo en cuenta el código del proceso y el valor de las entradas en los instantes de tiempo en que hay eventos en las señales de entrada del cronograma.

Tiempo de Simulación	ctrl	Dato_in	Dato_out

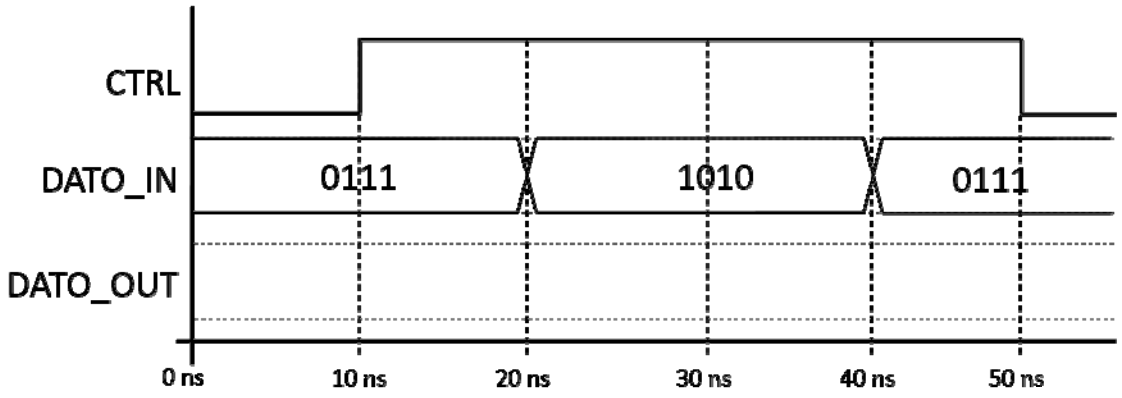


2. El siguiente código corresponde a un modelo INCORRECTO de **inv_trans**. Al realizarlo se ha omitido la entrada **ctrl** de la lista de sensibilidad, por lo que el proceso sólo se ejecutará en una simulación cuando haya eventos en **Dato_in**. Teniendo esto en cuenta, rellene la tabla adjunta y complete el cronograma de la figura.

```
process(Dato_in)
begin
  if ctrl = '0' then
    Dato_out <=  not Dato_in;

  else
    Dato_out(0) <= Dato_in(3);
    Dato_out(1) <= Dato_in(2);
    Dato_out(2) <= Dato_in(1);
    Dato_out(3) <= Dato_in(0);
  end if;
end process;
```

Tiempo de Simulación	ctrl	Dato_in	Dato_out



Como consecuencia del error cometido, el modelo no se corresponde con el de un circuito combinacional, si no con el de un *latch* (un combinacional realimentado), un sistema secuencial asíncrono. En el cronograma anterior puede observarse que el sistema modelado *tiene memoria* por el hecho de que la salida para una misma combinación de las entradas es diferente en distintos instantes de tiempo.

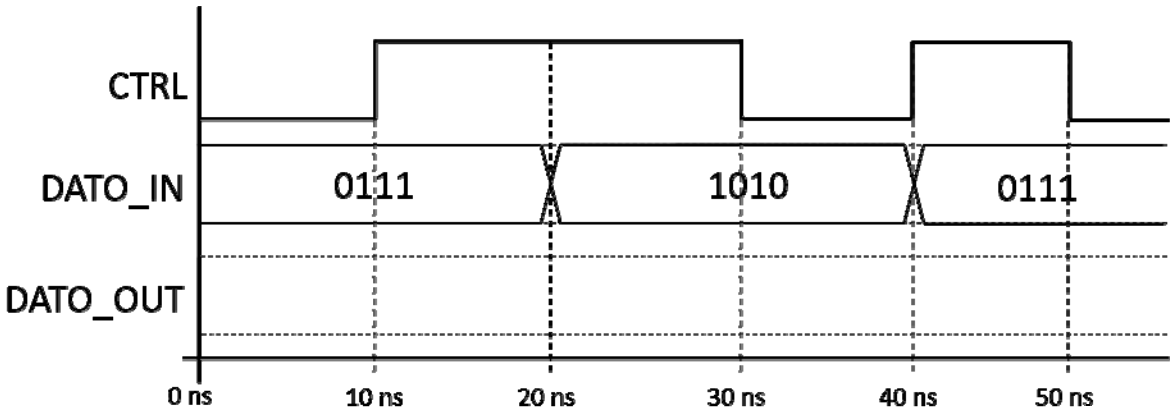
3. Identifique y consigne aquí los intervalos de tiempo de simulación en que a una misma combinación de entrada el modelo responde con diferentes combinaciones en la salida

4. El siguiente código corresponde a OTRO modelo INCORRECTO de **inv_trans**. En este caso se ha olvidado indicar el valor que toma la salida **Dato_out(3)** cuando **ctrl** vale 1. Teniendo esto en cuenta, rellene la tabla adjunta y complete el cronograma de la figura.

```
process(ctrl, Dato_in)
begin
  if ctrl = '0' then
    Dato_out <=  not Dato_in;

  else
    Dato_out(0) <= Dato_in(3);
    Dato_out(1) <= Dato_in(2);
    Dato_out(2) <= Dato_in(1);
  end if;
end process;
```

Tiempo de Simulación	ctrl	Dato_in	Dato_out



Como consecuencia del error cometido, nuevamente ocurre que el modelo corresponde a un circuito secuencial asíncrono.

5. Demuestre la corrección de la afirmación anterior en base al análisis de los resultados de la simulación representados en el cronograma.

Ejercicio 5

A continuación se resumen las reglas básicas de modelado de sistemas combinacionales que se han revisado y empleado en los ejercicios 2, 3 y 4 de esta actividad.

El modelo VHDL de la interfaz de un sistema se realiza construyendo una Declaración de Entidad, en la que se especifica:

1. El nombre del sistema modelado.

2. Las entradas y salidas (los puertos) de la interfaz del sistema, indicando su nombre, dirección y tipo de datos –el tipo de datos define implícitamente el número de bits de cada *puerto*.

3. En los modelos VHDL sintetizables la Declaración de entidad viene precedida por dos *cláusulas de visibilidad* que permiten emplear los tipos de datos *std_logic* y *std_logic_vector*:

```
library ieee;
use ieee.std_logic_1164.all;
```

El modelo de funcionamiento VHDL de un sistema digital se codifica en un Cuerpo de Arquitectura. El funcionamiento de los sistemas combinacionales puede expresarse empleando un *proceso con lista de sensibilidad*; estos procesos contienen sentencias que se ejecutan secuencialmente, durante el transcurso de una simulación, cada vez que hay *eventos* (cambios de valor) en cualquiera de las señales que forman parte de la *lista de sensibilidad* del proceso.

Para construir el código de un proceso que modele el funcionamiento de un sistema combinacional hay que tener en cuenta que:

- ✓ El proceso tiene que ser sensible a TODAS las entradas del combinacional.

- ✓ Dentro del proceso debe codificarse un algoritmo de ejecución secuencial que calcule, para TODAS las combinaciones de los bits de entrada, el valor que toma cada bit de salida.

Si se incumple cualquiera de las reglas anteriores, el modelado del sistema combinacional será INCORRECTO, ya que el proceso modelará el funcionamiento de un sistema secuencial asíncrono.

1. Si tiene dudas sobre la aplicación o el sentido de cualquiera de las reglas anteriores, debe consultar dichas dudas con su profesor; en caso contrario, o una vez resueltas las dudas que puedan haberle surgido, conteste a los siguientes apartados de este ejercicio.

2. Analice el código de los siguientes procesos y justifique si modela correctamente el funcionamiento que se indica:

   a) Un sistema combinacional que compara dos datos, A y B, y activa una salida cuando los datos son iguales

```
process(A, B)
begin
  if A = B then
    A_igual_B <= '1';
  end if;
end process;
```

b) Un multiplexor de dos canales

```
process(A, B)
begin
  if dir = '0' then
    S <= A;

  else
    S <= B;
  end if;
end process;
```

c) Un sistema combinacional que compara dos bits y activa una salida cuando son distintos

```
process(A, B)
begin
  A_distinto_B <= A xor B;

end process;
```

3. Corrija los errores que pudieran haberse cometido en la realización de los procesos del apartado anterior.

a)
b)
c)

4. Escriba el código de las Declaraciones de Entidad correspondientes a los sistemas modelados por los procesos anteriores

a)
b)
c)

5. Solución del Ejercicio 1

1. Complete la tabla de verdad adjunta para que represente el funcionamiento de un decodificador BCD a 7 segmentos con las salidas activas a nivel alto

D	C	B	A	S_A	S_B	S_C	S_D	S_E	S_F	S_G
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1	1	0	1	1	0	1	1
0	1	1	0	1	0	1	1	1	1	1
0	1	1	1	1	1	1	0	0	0	0
1	0	0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	0	0	1	1
Resto				X	X	X	X	X	X	X

2. Exprese en hexadecimal el valor de las salidas del decodificador BCD a 7 segmentos para cada uno de los dígitos decimales.

Ejemplo:

D	C	B	A	S_A	S_B	S_C	S_D	S_E	S_F	S_G
0	0	0	0	1	1	1	1	1	1	0



DÍGITO	0	1	2	3	4	5	6	7	8	9
HEXA	0x7E	0x30	0x6D	0x79	0x33	0x5B	0x5F	0x70	0x7F	0x73

3. Extraiga de la tabla de verdad:

   a. La forma canónica disyuntiva de S_E:

   $$S_E = \overline{D} \cdot \overline{C} \cdot \overline{B} \cdot \overline{A} + \overline{D} \cdot \overline{C} \cdot B \cdot \overline{A} + \overline{D} \cdot C \cdot B \cdot \overline{A} + D \cdot \overline{C} \cdot \overline{B} \cdot \overline{A}$$

   b. La expresión de S_G como producto de maxitérminos (forma canónica conjuntiva):

   $$S_G = (D + C + B + A) \cdot (D + C + B + \overline{A}) \cdot (D + \overline{C} + \overline{B} + \overline{A})$$

4. Obtenga de la tabla de verdad del decodificador las siguientes funciones lógicas en su expresión resumida:

   $$S_F = \sum_{(D,C,B,A)} (0,4,5,6,8,9)$$

   $$S_D = \prod_{(D,C,B,A)} (1,4,7,9)$$

5. Averigüe a qué salidas del decodificador corresponden las siguientes funciones lógicas:

   $F_1 = \overline{C} + \overline{A} \oplus B$ corresponde a la salida S_B

   $F_2 = B + D + \overline{A \oplus C}$ corresponde a la salida S_A

6. En los apartados anteriores ha extraído la expresión correspondiente a 4 salidas e identificado, a partir de expresiones dadas, las de otras dos. Obtenga, para la salida que resta, la expresión algebraica con menor número de términos y simplifíquela aprovechando que la tabla de verdad del decodificador es incompleta (contiene indiferencias).

   Queda S_C:

   $$S_C = C + \overline{B} + A$$